РОЗРОБКА МУЛЬТИМЕДІЙНИХ ЗАСТОСУНКІВ

Спеціальність 122 Комп'ютерні науки · група КН-3/1 · семестр I (V)

ЛАБОРАТОРНА РОБОТА 10

Unity: бюджет текстур

Тема 5 · 2 години · Unity

1. Про що ця робота

Усе, що ви робили досі, мало одну правильну відповідь. Ця робота влаштована інакше: правильних відповідей тут кілька, і головне в ній — не порахувати, а обґрунтувати.

Задача формулюється так, як її формулюють у житті. Є сцена. Є обмеження відеопам'яті, яке ставить платформа. Треба вкластися — і пояснити, чому ви зламали саме те, що зламали.

Арифметика тут проста, і ви впораєтеся з нею за двадцять хвилин. Складне починається там, де числа закінчуються: скільки роздільності віддати за формат, яку текстуру лишити недоторканою і що саме ви готові погіршити, щоб інше лишилося добрим.

Що ви зробите за пару

1. Порахуєте, скільки пам'яті займає сцена зараз.
2. Складете план: який формат і яку роздільність дати кожній текстурі.
3. Застосуєте план в Unity й перевірите результат у профайлері.
4. Поміряєте, що саме зламалося, і напишете обґрунтування кожного рішення.

> **Важливо** Числа в цій роботі стосуються роздільності й формату — їх можна порахувати на папері. Усе, що показує профайлер Unity, ви міряєте самі: ці значення залежать від версії, платформи та налаштувань проєкту, і наводити їх як контрольні було б неправильно.

2. Що треба знати

2.1. Формула, і в ній усе

Пам'ять текстури не залежить від того, що на ній намальовано. Вона залежить лише від трьох чисел:

```
байтів = ширина × висота × (біт на піксель) / 8

з міпмапами множимо ще на 4/3   (додаткова третина)
```

Біти на піксель задає формат, і це єдине, що міняється від формату до формату:

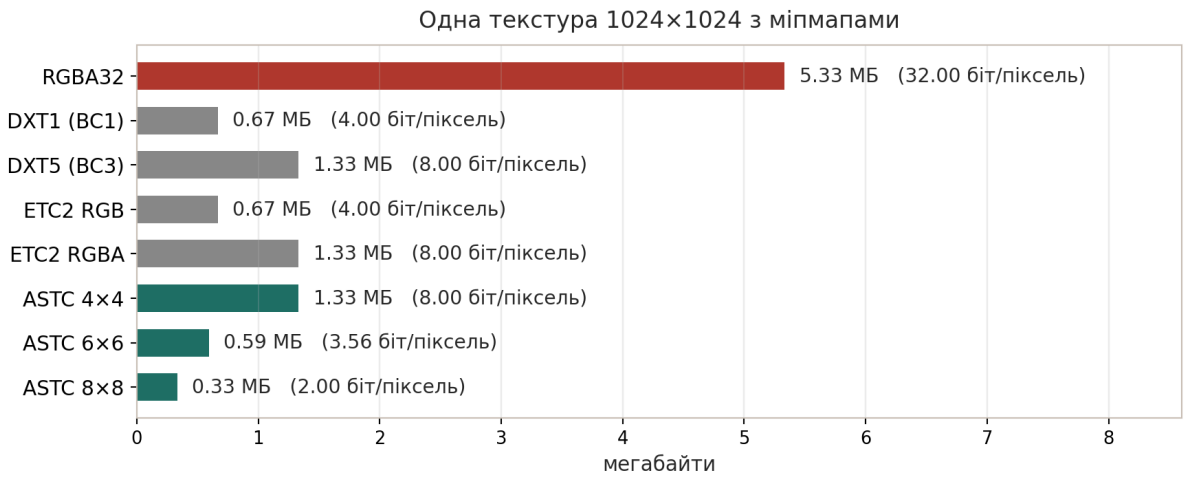


Рисунок 1 — Скільки займає одна текстура 1024×1024 з міпмапами

Формат	Біт/пкс	1024², МБ	Коли брати
RGBA32 — без стиснення	32	5,33	майже ніколи; лише коли все інше зламалося
ASTC 4×4	8	1,33	нормалі, інтерфейс, усе чутливе
ASTC 6×6	3,56	0,59	звичайний колір — робочий варіант за замовчуванням
ASTC 8×8	2	0,33	далеке тло, службові текстури
ETC2 RGB / RGBA	4 / 8	0,67 / 1,33	старіші пристрої без підтримки ASTC
DXT1 / DXT5 (BC1 / BC3)	4 / 8	0,67 / 1,33	настільні платформи

Зверніть увагу Зменшення сторони вдвічі дає рівно те саме, що й учетверо менший формат: пікселів стає вчетверо менше. Тому роздільність і формат — це два важелі, які тягнуть в один бік, і їх можна міняти один на одного.

2.2. Сцена, з якою працюємо

Дванадцять текстур, усі поки що без стиснення. Бюджет відеопам'яті на текстури — 24 МБ.

Текстура	Розмір	Роль
Ground_Albedo	2048²	колір землі
Ground_Normal	2048²	нормалі землі
Building_Albedo	2048²	колір будівель
Building_Normal	2048²	нормалі будівель
Props_Albedo / Props_Normal	1024²	дрібні об'єкти
Skybox	2048²	небо
Character_Albedo / Character_Normal	1024²	персонаж

Текстура	Розмір	Роль
UI_Atlas	2048²	інтерфейс: текст, кнопки, іконки
Decals	1024²	наклейки з прозорістю
Noise	512²	службовий шум

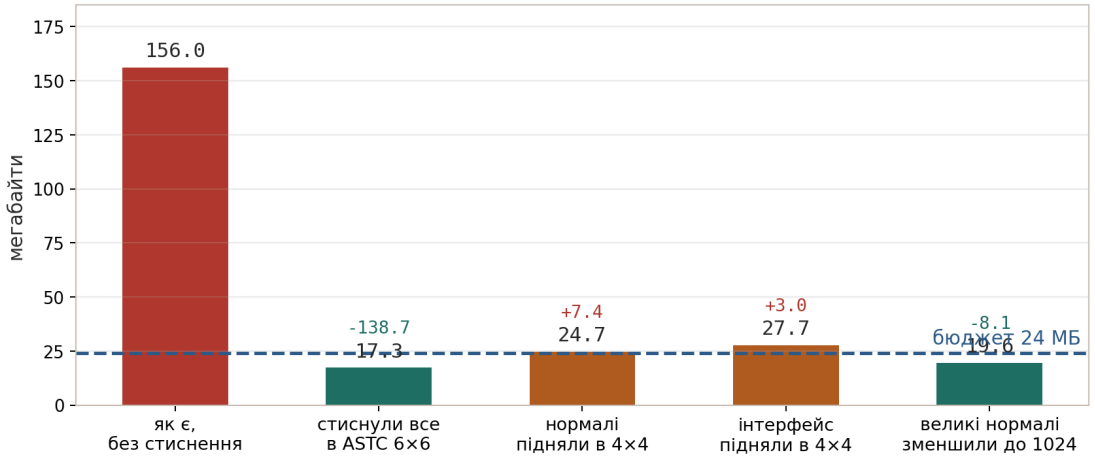
Разом без стиснення — 156 МБ. Тобто перевищення бюджету більш ніж у шість разів.

2.3. Чому не можна просто стиснути все

Перша думка очевидна: поставити всім ASTC 6×6 і закрити питання. Арифметично це спрацює — вийде 17,3 МБ, із великим запасом. Візуально зламаються дві речі.

- Нормалі. У лабораторній 9 ви поміряли, що блокове стиснення відхиляє напрямок нормалі на кілька градусів. Освітлення по такій поверхні рахується неправильно скрізь.
- Інтерфейс. Різкі межі літер і піктограм — найгірший випадок для будь-якого блокового формату.

Тому шлях до бюджету не прямий. Два кроки посередині повертають пам'ять назад — і саме вони роблять цю роботу роботою, а не вправою на множення.



Шлях не прямий: два кроки посередині повертають пам'ять назад, бо сліпе стиснення зіпсувало нормалі й інтерфейс.

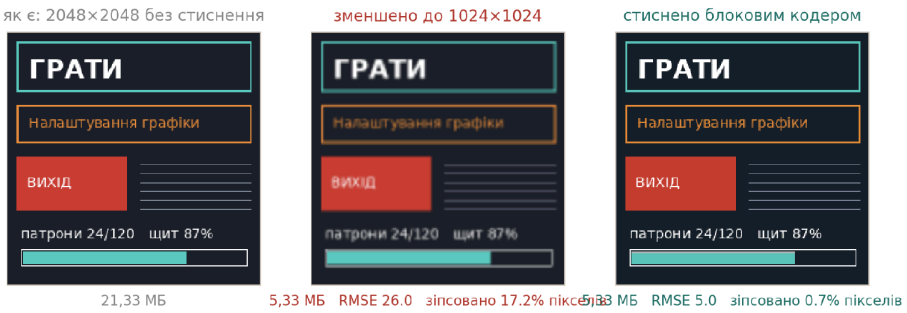
Рисунок 2 — Як рухається бюджет, крок за кроком

2.4. Найцікавіше рішення — інтерфейс

UI-атлас 2048×2048 без стиснення коштує 21,33 МБ — майже весь бюджет на одну текстуру. Зменшити його треба вчетверо, і способів рівно два.

- Зменшити роздільність до 1024×1024, лишивши без стиснення — 5,33 МБ.
- Лишити роздільність 2048, але стиснути в ASTC 4×4 — теж 5,33 МБ.

Пам'ять однакова до байта. Отже, вибір робиться не за розміром, а за тим, що саме зламається. Більшість, пам'ятаючи лабораторну 9, вибирає перший варіант — і помиляється.



Обидва варіанти коштують однаково — 5,33 МБ. Але зменшення роздільності псує вп'ятеро більше: тонкі штрихи літер зникають безповоротно, а блокове стиснення на інтерфейсі майже не помиляється, бо в його блоках зазвичай лише два кольори — тло й літера.

Рисунок 3 — Два способи вчетверо здешевити інтерфейс

Виміряно на тестовому атласі: зменшення роздільності дає RMSE 26,0 і псує 17,2 % пікселів, блокове стиснення — RMSE 5,0 і 0,7 %. Різниця у п'ять разів, і не на користь інтуїції.

> **Чому так** Блокове стиснення ламається тоді, коли в одному блоці 4×4 зустрічаються три різні відтінки — це ви й бачили в лабораторній 9. В інтерфейсі такого майже не буває: у блоці зазвичай лише тло й літера, тобто два кольори, а з двома кольорами формат справляється точно. Зменшення ж роздільності знищує тонкі штрихи літер безумовно, незалежно від того, скільки там кольорів.

3. Хід роботи

Сцену видає викладач. Усі згадані далі пункти меню можуть називатися трохи інакше залежно від версії Unity — орієнтуйтеся на зміст, а не на точні слова.

Завдання 1. Порахувати, що є (20 хвилин)

Спершу на папері, без Unity. Напишіть маленький калькулятор — він знадобиться в кожному наступному завданні.

```
BPP = {'RGBA32': 32, 'ASTC 4×4': 8, 'ASTC 6×6': 128/36, 'ASTC 8×8': 2}

def mem(side, fmt, mips=True):
    b = side * side * BPP[fmt] / 8
    if mips:
        b *= 4/3
    return b / 2**20            # мегабайти

scene = [('Ground_Albedo', 2048), ('Ground_Normal', 2048), ...]
print(sum(mem(s, 'RGBA32') for _, s in scene))
```

Число 128/36 для ASTC 6×6 береться так: блок 6×6 пікселів завжди кодується 128 бітами, отже на піксель припадає 128/36 ≈ 3,56 біта.

> **Як перевірити** Для наведеної сцени має вийти рівно 156,00 МБ. Якщо вийшло 117 — забули множник 4/3 на міпмапи.

Тепер відкрийте сцену в Unity й звірте. Пам'ять кожної текстури видно внизу вікна Inspector, коли текстуру виділено. Загальну цифру дає вікно Profiler у розділі Memory.

Завдання 2. Скласти план (20 хвилин)

Пройдіть по текстурах і для кожної випишіть формат і роздільність разом із причиною. Правила, які випливають із того, що ви вже міряли:

Що за текстура	Що ставимо	Чому
звичайний колір	ASTC 6×6	найдешевший формат, на фотографічних поверхнях не помітно
нормалі	ASTC 4×4	кутова похибка з ЛР 9; економити краще роздільністю
інтерфейс	ASTC 4×4, роздільність не чіпати	у блоках по два кольори; формат дешевший за зменшення
далекі поверхні	менша роздільність	гравець не підходить, деталі все одно губляться
службові текстури	ASTC 8×8 або менший розмір	якість не впливає ні на що

Проженіть план через калькулятор. Якщо не вкладається — шукайте, де ще зменшити роздільність, а не де ще сильніше стиснути.

> **Контрольна точка** Один із правильних планів дає 19,59 МБ при бюджеті 24. Ваш може відрізнятися — головне, щоб кожне рішення мало причину, записану словами.

Завдання 3. Застосувати в Unity (30 хвилин)

Для кожної текстури в Inspector:

1. Увімкніть вкладку потрібної платформи (Android або iOS) — у ній є власні налаштування, окремі від стандартних.
2. Поставте Max Size — це і є роздільність, якою текстура потрапить у збірку.
3. Виберіть Format із вашого плану.
4. Для нормалей переконайтеся, що Texture Type стоїть Normal map.
5. Для інтерфейсу й усього, що не є кольором, зніміть прапорець sRGB.
6. Натисніть Apply і подивіться на рядок внизу Inspector — там з'явиться новий розмір.

> **Обережно** Прапорець sRGB — той самий, про який ішлося на лекції. Для кольору він має бути ввімкнений, для нормалей, масок і шуму — вимкнений. Переплутаєте — освітлення поїде, а пам'ять не зміниться ні на байт.

Якщо потрібного формату немає у списку — платформа його не підтримує. Для Android без ASTC беріть ETC2; тоді числа будуть інші, і план доведеться перерахувати.

Завдання 4. Перевірити й обґрунтувати (20 хвилин)

Відкрийте Profiler, розділ Memory, і знайдіть рядок текстур. Запишіть цифру до й після.

Потім пройдіться по сцені очима. Шукайте конкретно:

- чи читається дрібний текст в інтерфейсі;
- чи не з'явилися плями або смуги на поверхнях із нормалями;
- чи не стало видно квадратів 4×4 на різких межах;

- чи не потемнішали віддалені поверхні — це ознака переплутаного прапорця sRGB.

Зробіть два скріншоти однієї ділянки — до і після — і складіть таблицю обґрунтувань: текстура, що змінили, скільки зекономили, що зіпсувалося.

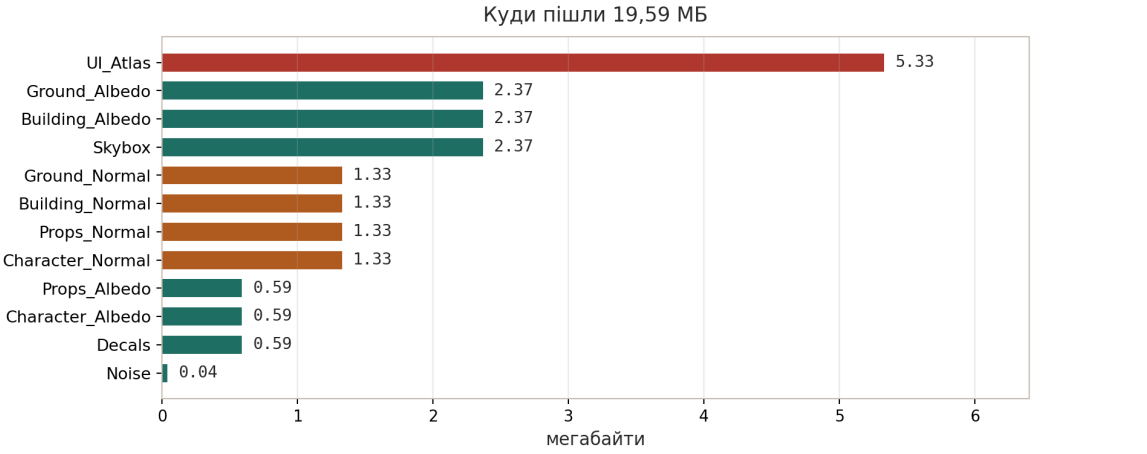


Рисунок 4 — Розкладка одного з правильних планів

Зверніть увагу на верхній рядок. Навіть стиснутий інтерфейс лишається найдорожчою текстурою сцени — просто тому, що він єдиний зберіг повну роздільність. Це нормально: він єдиний, який гравець розглядає впритул увесь час.

4. Перевірте перед здачею

- ☐ Калькулятор дає 156,00 МБ для початкового стану сцени.
- ☐ План складено для всіх дванадцяти текстур, у кожної записано причину.
- ☐ Порахований бюджет укладається в 24 МБ.
- ☐ Налаштування застосовані в Unity, Profiler показує зменшення.
- ☐ Зроблено скріншоти до і після тієї самої ділянки сцени.
- ☐ Дрібний текст в інтерфейсі читається, нормалі не плямисті.

5. Що робити, якщо не працює

Симптом	Причина	Як виправити
Порахували 117 МБ замість 156	Забуто множник 4/3 на міпмапи	Міпмапи додають третину; їх вмикають майже завжди
Профайлер показує більше, ніж ви порахували	Є текстури, яких немає у вашому списку	Lightmap, Reflection Probe і шрифти теж займають пам'ять
Зміна формату нічого не дала	Правили стандартну вкладку, а не вкладку платформи	Налаштування платформи перекривають стандартні
Потрібного формату немає у списку	Платформа його не підтримує	Для старих Android беріть ETC2 і перерахуйте план
Поверхні потемнішали після зміни	Переплутано прапорець sRGB	Колір — sRGB увімкнено; нормалі, маски, шум — вимкнено

Симптом	Причина	Як виправити
Нормалі дали плями й смуги	Texture Type не стоїть Normal map	Unity кодує нормалі інакше, якщо знає, що це нормалі
Інтерфейс розплився	Зменшили роздільність замість стиснення	Поверніть 2048 і поставте ASTC 4×4 — коштує стільки ж

6. Запитання на здачі

1. Чому пам'ять текстури не залежить від того, що на ній намальовано?
2. Зменшити сторону вдвічі чи взяти формат із учетверо меншим числом біт — що дешевше? Покажіть на числах.
3. Назвіть дві текстури, для яких ви НЕ взяли найдешевший формат, і поясніть чому.
4. Інтерфейс можна здешевити вчетверо двома способами за однакову ціну. Який ви обрали і на підставі яких чисел?
5. Чому для нормалей економити краще роздільністю, а не форматом?
6. Що саме робить прапорець sRGB і до яких текстур його не можна вмикати?
7. Бюджет зменшили вдвічі — до 12 МБ. Що ви відріжете першим і чому саме це?

7. Як оцінюється

Що саме	Балів
Початковий бюджет пораховано правильно (завдання 1)	2
План складено для всіх текстур із причинами (завдання 2)	3
Налаштування застосовані, сцена вкладається в бюджет (завдання 3)	3
Результат перевірено в профайлері й очима, обґрунтування написане (завдання 4)	2
Разом	10

8. Що здати в Classroom

- Таблиця обґрунтувань: текстура, було, стало, скільки зекономлено, що зіпсувалося.
- Скріншоти профайлера до і після.
- Два скріншоти однієї ділянки сцени — до і після.
- Код калькулятора бюджету.

Форма контролю — усне опитування за запитаннями розділу 6. Головна вимога на захисті: захистити кожне рішення числом або спостереженням, а не словами «так здалося краще».

Якщо залишився час

Перерахуйте той самий план під платформу без підтримки ASTC — лише ETC2. Нормалі там доведеться класти в ETC2 RGB по 4 біти на піксель, а прозорість — у ETC2 RGBA по 8.

Складіть новий план, вкладіться в ті самі 24 МБ і запишіть, чим довелося пожертвувати додатково. Це рівно та робота, яку виконують, коли гру портують на старіші пристрої.